

第 3 周 实验报告

课程：系统开发工具基础

姓名：朱英俊 学号：24070030103

提交日期：2026 年 9 月 7 日

目录

- 实验概览
- 第 9 题：从源码构建并在干净环境安装 Wheel
- 第 10 题：让编程智能体进入可验证的修复循环
- 第 11 题：把协作材料改成可执行信息
- 第 12 题：修复一个可复现的线性回归训练循环
- 课后练习与解题感悟（14 个实例）
- 版本控制与提交记录
- 小结

1 实验概览

本周授课内容为「Packaging and Shipping Code」「智能体编程」「不止于代码」「Python 与 PyTorch」，对应四道实验题：q09 用 pyproject.toml + src 布局把最小问候包 greetlab 构建成 wheel 并在干净 venv 中安装运行；q10 让编程智能体进入「失败测试 -> 修复 -> 复跑」的可验证循环；q11 把三段质量较差的协作材料重写为可执行信息；q12 补全 PyTorch 线性回归训练循环并验证收敛。

q09 打包与交付	pyproject.toml / build / venv	greetlab_*.whl
q10 智能体编程	pytest / 编程智能体	tests/、ai_log.md
q11 不止于代码	Markdown 协作规范	communication.md
q12 Python 与 PyTorch	torch / nn.Linear / SGD	train.py

报告每个实验按「任务说明 -> 操作步骤 -> 运行结果 -> 要点」组织；运行结果均来自 Windows + Git Bash 沙箱中的真实执行。

2 第 9 题：从源码构建并在干净环境安装 Wheel

任务说明

在 q09 中创建最小 Python 命令行包 greetlab，用 python -m build 从源码生成 wheel；新建干净虚拟环境，仅从 wheel 安装（不联网），并切换到 q09 之外运行 sdt-greet --name <学号> 验证输出。

操作步骤

1) 创建 src 布局：src/greetlab/__init__.py + cli.py，并补齐 pyproject.toml，把「学号」替换为 24070030103。

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-24070030103"
version = "0.1.0"
requires-python = ">=3.9"

[project.scripts]
sdt-greet = "greetlab.cli:main"
```

```
[tool.setuptools.packages.find]
where = ["src"]
```

其中 [project.scripts] 把 sdt-greet 命令映射到 greetlab.cli:main，是安装后能直接调用的关键。

2) 运行 python -m build 生成 wheel:

```
$ cd q09
$ python -m build
...
Successfully built greetlab_24070030103-0.1.0.tar.gz
and greetlab_24070030103-0.1.0-py3-none-any.whl
```

3) 新建干净 venv，仅从 wheel 安装 (--no-index 关闭 PyPI, --find-links 只查本地):

```
$ python -m venv clean-venv
$ clean-venv/Scripts/pip install --no-index --find-links dist greetlab-24070030103
Looking in links: dist
Successfully installed greetlab-24070030103-0.1.0
```

4) 切换到 q09 之外运行:

```
$ cd ..
$ ./q09/clean-venv/Scripts/sdt-greet.exe --name 24070030103
Hello, 24070030103!
$ ./q09/clean-venv/Scripts/sdt-greet.exe --name 朱英俊
Hello, 朱英俊!
```

运行结果

build 同时产出 sdist(.tar.gz) 与 wheel(.whl, py3-none-any 表示纯 Python 跨平台); 干净 venv 安装成功, pip show 显示 Version: 0.1.0 且 Requires 为空; 在 q09 之外运行 sdt-greet --name 24070030103 得到 Hello, 24070030103!。

要点

* src 布局 + packages.find where=["src"] 是现代 Python 打包推荐结构。

* --n

3 第 10 题：让编程智能体进入可验证的修复循环

任务说明

复制 q09 为 q10，添加先失败的测试：--name 只含空白字符时 main 应以 SystemExit(2) 结束；向智能体说明目标、约束与测试命令，要求它修改实现并复跑；人工检查 diff、撤销无关修改并复跑，把记录写入 ai_log.md (<=5 行)。

操作步骤

1) 添加 tests/test_cli.py (monkeypatch 注入 sys.argv)：

```
def test_blank_name_exits_code_2(monkeypatch):
    monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", ""])
    with pytest.raises(SystemExit) as excinfo:
        main()
    assert excinfo.value.code == 2
```

2) 先跑测试确认失败（红灯）：

```
$ PYTHONPATH=src pytest tests/test_cli.py -v
test_blank_name_exits_code_2 FAILED
Failed: DID NOT RAISE SystemExit
Captured stdout: Hello, !
```

3) 向智能体说明「目标：空白姓名退出码 2；约束：保留 argparse、用 p.error 退出；测试命令：上面的

pytest」。智能体新增两行校验：

```
a = p.parse_args()
if not a.name.strip():
    p.error("--name 不能只含空白字符")
print(f"Hello, {a.name}!")
```

4) 人工 git diff 确认仅 +2 行、无无关改动，复跑测试（绿灯）：

```
$ PYTHONPATH=src pytest tests/test_cli.py -v
2 passed in 0.07s
```

ai_log.md 共 5 行，记录核心提示、智能体改动与人工验证。

要点

* 「可验证循环」= 红灯测试 -> 明确契约 -> 智能体改码 -> 人工 diff 审查 -> 绿灯。

* arg

4 第 11 题：把协作材料改成可执行信息

任务说明

围绕「空姓名仍输出问候语」缺陷，把 Issue / 提交信息 / 评审意见三段材料重写为可执行信息写入 communication.md，全文 <=400 字。

改写前 vs 改写后

改写前：

Issue: Windows 上运行不了，尽快修复。
提交信息: fix bug
评审意见: 这里写得不好，重写。

改写后（节选）：

Issue 标题: sdt-greet --name " " 对空白姓名仍输出问候语并以 0 退出
环境: Windows（版本待确认）；greetlab-24070030103 v0.1.0；Python 待确认
复现命令: sdt-greet --name " "
期望结果: 以 SystemExit(2) 结束，不输出问候语
实际结果: 输出 Hello, !，退出码 0

提交信息: fix: 空白姓名改为报错退出，而非输出问候语
正文: --name 传入纯空白时仍打印 Hello, ! 并返回 0。新增 strip 校验使其以 2 退出

评审意见:
Blocking: cli.py 缺少空白姓名校验，会输出无意义 Hello, ! 且退出码 0 误判成功
Nit: 可在 --name 的 help 文本补充「不可为空或纯空白」说明

要点

* 可执行的 Issue 必须有复现命令与期望/实际结果；未知信息标注「待确认」。

* 提交

5 第 12 题：修复一个可复现的线性回归训练循环

任务说明

补全 train.py 的训练循环 TODO: 正确调用 zero_grad / backward / step；训练后 model.eval() + torch.no_grad() 打印最终损失、weight、bias。要求损失 < 0.001，且不得把 weight/bias 直接赋值为 3 和 -1。

操作步骤

数据 $y = 3x - 1$ ，模型 nn.Linear(1, 1)，损失 MSE，优化器 SGD(lr=0.1)，训练 200 轮：

```
for _ in range(200):
    pred = model(x)
```

```

    loss = loss_fn(pred, y)
    opt.zero_grad()    # 清空梯度
    loss.backward()     # 反向传播
    opt.step()          # 更新参数

model.eval()
with torch.no_grad():
    pred = model(x)
    final_loss = loss_fn(pred, y)
    print(f"final loss = {final_loss.item():.6f}")
    print(f"weight = {model.weight.item():.6f}, bias = {model.bias.item():.6f}")

```

运行结果

```

$ python train.py
final loss = 0.000000
weight = 2.999997, bias = -1.000000

```

最终损失 < 0.001 , weight 收敛到 2.999997、bias 收敛到 -1.000000, 与生成式 $y = 3x - 1$ 一致。

要点

* 训练循环五要素: zero_grad / backward / step / eval / no_grad, 缺一不可。

* tor

6 课后练习与解题感悟（14 个实例）

练习1 查看 wheel 内部结构

python -m zipfile -l dist/*.whl 列出代码、METADATA、entry_points.txt、RECORD, wheel 本质是带元数据的 zip。

练习2 sdist 与 wheel 产物差异

python -m build 一次产出 .whl 与 .tar.gz, 前者可直装、后者需再构建。

练习3 editable 与 wheel 安装区别

pip install -e . 开发态即时生效; pip install dist/*.whl 安装固定快照。

练习4 pyproject 入口点脚本

sdt-greet --help 显示 usage: sdt-greet [-h] --name NAME, 入口点让包变成命令。

练习5 修复循环提示词模板

目标/约束/测试命令 三要素显式化验收标准, 是智能体可验证的关键。

练习6 git diff 审查智能体改动

git diff 显示智能体只新增两行校验, 无无关改动。

练习7 撤销无关修改

git restore 精确回滚单文件, 审查后选择性回滚是标准动作。

练习8 Conventional Commits

fix: / feat(scope): 格式让提交历史可读、可筛选、可生成 CHANGELOG。

练习9 结构化 Issue 模板

环境/复现/期望/实际 四要素, 缺失复现步骤的缺陷报告约等于无效。

练习10 祈使语气提交信息

标题说明做了什么, 正文补充动机与方案, 面向未来读历史的人。

练习11 Review 分级标注

Blocking 必须改 / Suggestion 建议改 / Nit 可选改, 让作者分清优先级。

练习12 张量基础运算

torch.linspace(-1, 1, 5) 得到 5 个等间距点, shape=[5]、dtype=float32, reshape 只改形状。

练习13 训练循环五要素拆解

zero_grad 防梯度累加、backward 求导、step 更新、eval 评估态、no_grad 免梯度。

练习14 no_grad 梯度追踪对比

a.grad = tensor(4.) (d(a^2)/da) ; no_grad 内 requires_grad=False, 不建计算图。

感悟小结

14 个练习把四章内容织成一张网：Packaging 打通「源码 -> wheel -> 干净安装 -> 入口点调用」交付链路；智能体编程强调「可验证循环」；不止于代码把 Issue/提交信息/评审升维成可执行协作规范；PyTorch 用最小线性回归跑通训练五要素。最深体会：交付、协作、训练三者殊途同归，都要「可复现、可验证、可审查」。

7 版本控制与提交记录

仓库地址

GitHub: <https://github.com/yingjunzhu520/system-tools-hamesome-3>

提交记录（分次提交，非全量）

本地仓库按「初始化模板 -> q09 -> q10 -> q11 -> q12 -> 截图脚本 -> 报告 -> README -> 提交记录截图」的顺序分次提交，每步均为增量修改。详见报告正文的 git log --oneline 输出与 commit_screenshot.png。

8 小结

本周四题覆盖「打包交付 / 智能体编程 / 协作规范 / PyTorch」四条主线：q09 走通 src + pyproject + build 的完整打包链路；q10 跑通「失败测试 -> 智能体修复 -> diff 审查 -> 复跑」的可验证循环；q11 把协作材料重写为可执行信息；q12 补全训练循环五要素并验证损失 < 0.001。课后 14 个练习进一步把四章内容系统化。